Writing Classes by Hand

Fields, constructors, getters, and setters from a UML diagram

Today you're going to write several classes yourself, following the same pattern every time, until you can do it without looking.

The pattern

Every class on this sheet is written in the same order:

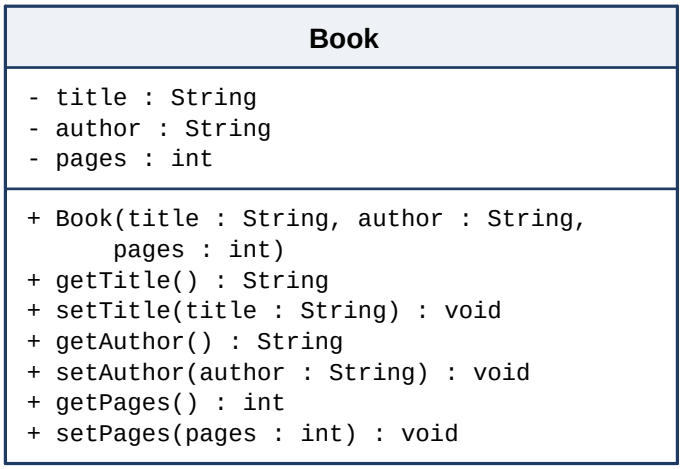
1. **Fields.** One line for each piece of data the class holds.
2. **Constructor.** One constructor that takes a value for every field and stores it.
3. **Getters and setters.** For each field, in the same order as the fields, write its getter and then its setter.

Why follow the pattern?

- **Everyone reads it the same way.** Almost every Java class you'll see on the job is laid out like this. When you open a class someone else wrote, you already know where the data is, where the constructor is, and where to find each getter and setter. They'll know the same thing about yours.
- **The names matter.** Java tools and libraries look for methods named exactly `getName`, `setName`, and `isActive`. Later in the course, you'll use tools that read and fill in your objects by calling these methods for you. If you name a getter `fetchName`, those tools won't find it.
- **Mistakes are easier to spot.** When every class follows the same order, a missing setter or a field the constructor forgot to store stands out right away.

Reading a UML class diagram

A UML class diagram is a box with three sections. Here's the one for the example class, **Book**:



Top: the class name.

Middle: the fields. Each one is written `name : Type`.

Bottom: the constructor and methods. A method's return type comes after the colon at the end. The constructor has the same name as the class and no return type.

What do `-` and `+` mean? A minus sign means **private** and a plus sign means **public**. So fields get the word `private` in your code, and the constructor and methods get `public`. We'll talk about why next week.

Example: Book

Here's the diagram from the previous page turned into Java. Compare each line of the diagram to the code it became.

```
public class Book {

    private String title;
    private String author;
    private int pages;

    public Book(String title, String author, int pages) {
        this.title = title;
        this.author = author;
        this.pages = pages;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getAuthor() {
        return author;
    }

    public void setAuthor(String author) {
        this.author = author;
    }

    public int getPages() {
        return pages;
    }

    public void setPages(int pages) {
        this.pages = pages;
    }
}
```

What's `this.title = title`? Inside the constructor and the setter, there are two things named `title`: the field and the parameter. `this.title` means the field that belongs to this object. Plain `title` means the parameter that was passed in. So the line says "store the value that was passed in, in this object's field." Don't worry if it feels strange at first. It will click after a few classes.

A note on real UML. In the real world, most UML diagrams leave the getters and setters out, because everyone assumes they're there. We're drawing them for now so you can see the whole pattern. Class 3 will show you what the shorter version looks like.

Prove it works

For every class you write, test it in `main`: create an object with the constructor, print each field with its getter, then use a setter to change something and print it again.

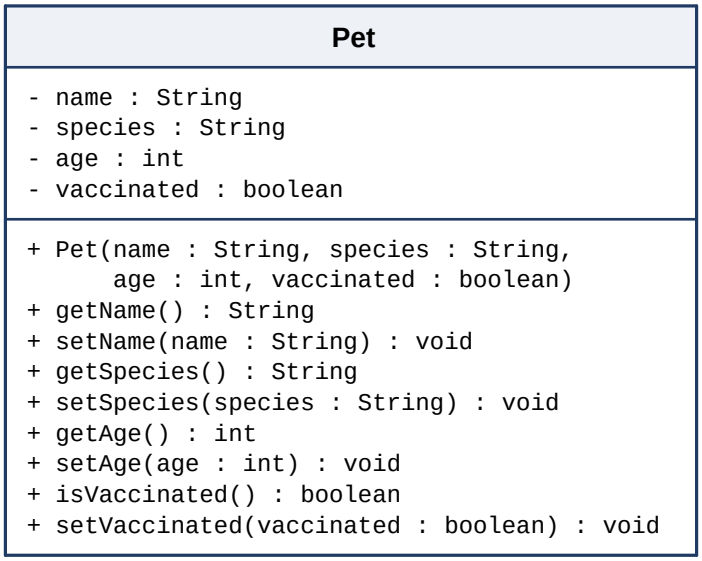
```
Book book = new Book("Dune", "Frank Herbert", 412);
System.out.println(book.getTitle());
System.out.println(book.getAuthor());
System.out.println(book.getPages());

book.setPages(896);
System.out.println(book.getPages());
```

Getting `null`, `0`, or `false`? If a getter prints one of those instead of the value you passed in, look at your constructor. You probably wrote `title = title` instead of `this.title = title`.

Class 1: Pet

Write the `Pet` class from this diagram, then prove it works in `main`.



Notice the boolean getter. The getter for a `boolean` starts with `is` instead of `get`, so it's `isVaccinated()`. The setter still starts with `set`.

Class 2: Student

Same pattern, a couple of new types.

Student
- name : String - letterGrade : char - gpa : float - enrolled : boolean
+ Student(name : String, letterGrade : char, gpa : float, enrolled : boolean) + getName() : String + setName(name : String) : void + getLetterGrade() : char + setLetterGrade(letterGrade : char) : void + getGpa() : float + setGpa(gpa : float) : void + isEnrolled() : boolean + setEnrolled(enrolled : boolean) : void

Testing tips. A `char` goes in single quotes: `'A'`. A `float` needs an `f` at the end: `3.5f`. Without the `f`, Java treats `3.5` as a `double` and won't compile.

Class 3: Shipment

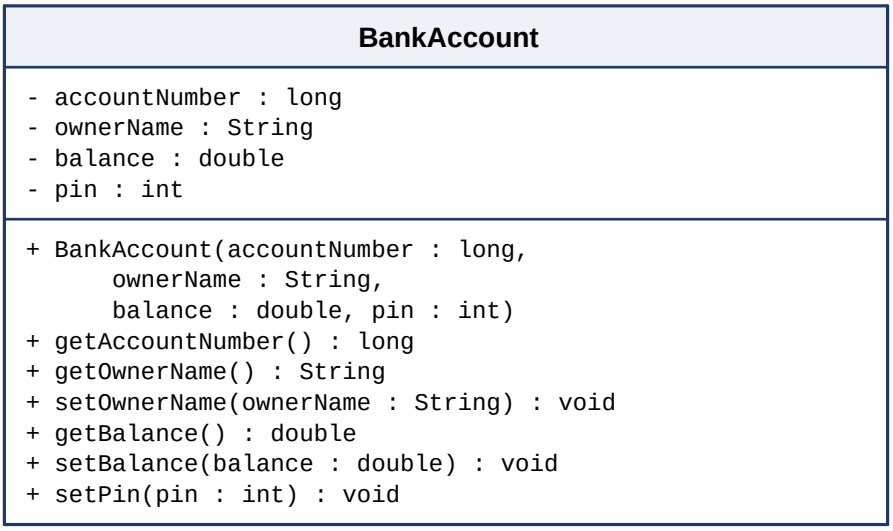
This diagram looks different. Read the note before you start.

Shipment
- trackingNumber : String - weightKg : double - distanceMiles : long - priority : char - delivered : boolean
+ Shipment(trackingNumber : String, weightKg : double, distanceMiles : long, priority : char, delivered : boolean)

Where did the getters and setters go? This is what most real UML diagrams look like. When the getters and setters aren't listed, everyone assumes every field has both. So you still write all of them, in the same order as always: fields, constructor, then a getter and setter for each field.

Class 4: BankAccount

This class breaks the pattern. When a class doesn't follow the usual pattern, the diagram lists its methods so you know exactly what to write. Write exactly what's listed, nothing more.



Look closely. `ownerName` and `balance` follow the normal pattern. But `accountNumber` has a getter and no setter, and `pin` has a setter and no getter.

Why would a real class do that? An account number is assigned once when the account is opened and should never change, so nothing gets to set it after the constructor. A PIN is the opposite. A customer should be able to change it, but no part of the program should be able to read it back out. Leaving out a getter or a setter is how a class protects its data. You'll see much more of this next week.

Testing tip. Account numbers are big, so the literal needs an `L` at the end to be a `long`: `1002003004L`. When you test, call `setPin(4321)`. You won't be able to print the PIN afterward, and that's the point.